Realistic Enemy AI in Stealth Games

Marta Pecharroman, Ander Fernandez & Suyay Charles

Team RRRMMMSSS

Table of contents

01 Group Navigation

Main Approaches

Flocking

Non very strict rules.
Every entity travels with similar speed and direction.
Not evenly distributed.

Formations

Very strict.
Even and balanced arrangements.

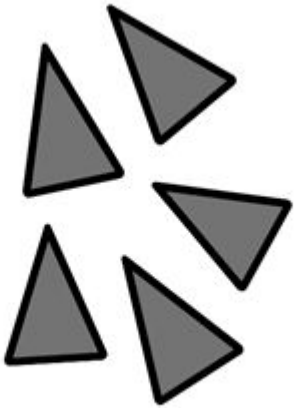
Social Groups

Defines hierarchies.
Takes care of aspects like dialogue inside the group.

Main Approaches

Flocking

Non very strict rules.
Every entity travels with similar speed and direction.
Not evenly distributed.



Formations

Very strict.
Even and balanced arrangements.

Social Groups

Defines hierarchies.
Takes care of aspects like dialogue inside the group.

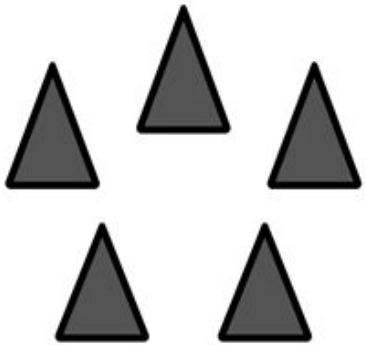
Main Approaches

Flocking

Non very strict rules.
Every entity travels with similar speed and direction.
Not evenly distributed.

Formations

Very strict.
Even and balanced arrangements.



Social Groups

Defines hierarchies.
Takes care of aspects like dialogue inside the group.

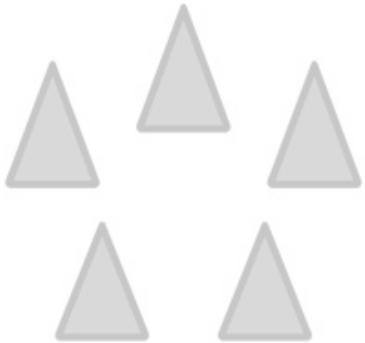
Main Approaches

Flocking

Non very strict rules.
Every entity travels with similar speed and direction.
Not evenly distributed.

Formations

Very strict.
Even and balanced arrangements.



Social Groups

Defines hierarchies.
Takes care of aspects like dialogue inside the group.

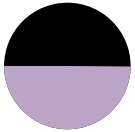
How do we approach Group Navigation?

How do we approach Group Navigation?

Problem statement

How do we get a realistic human motion simulation, which also avoids heavy computational cost?

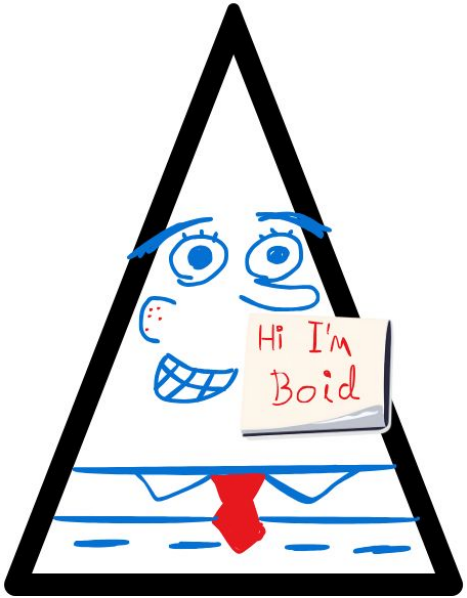
How do we approach Group Navigation?



Solution

Flocking
+
Hierarchies

Craig Reynold's *Boids*



Each entity of a flock is a 'Boid'

Craig Reynold's *Boids*

Alignment

Uses velocity of other boids to orientate

Cohesion

Steers boids towards the center using the average position

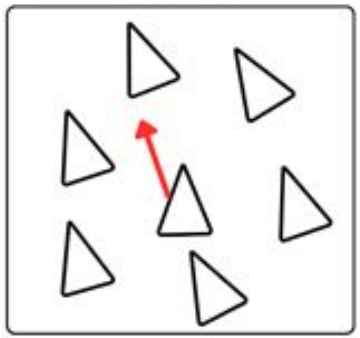
Separation

Gets direction vectors from other boids to separate from them

Craig Reynold's *Boids*

Alignment

Uses velocity of other boids to orientate



Cohesion

Steers boids towards the center using the average position

Separation

Gets direction vectors from other boids to separate from them

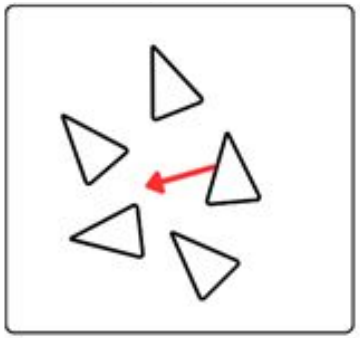
Craig Reynold's *Boids*

Alignment

Uses velocity of other boids to orientate

Cohesion

Steers boids towards the center using the average position



Separation

Gets direction vectors from other boids to separate from them

Craig Reynold's *Boids*

Alignment

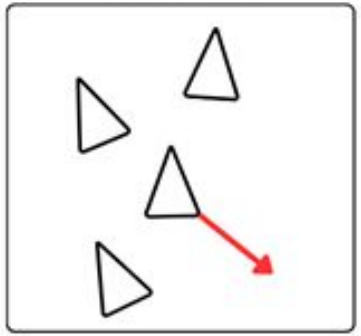
Uses velocity of other boids to orientate

Cohesion

Steers boids towards the center using the average position

Separation

Gets direction vectors from other boids to separate from them



Additional Forces

Obstacle Avoidance

Forces coming from walls, avoiding going through them.

Leader Seeking

The flock's leader has a stronger attraction weight than others.

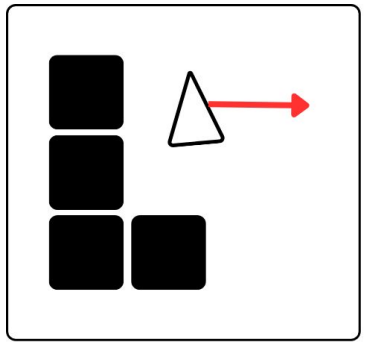
Inertia Preservation

Stabilizes steering and gets rid of randomness.

Additional Forces

Obstacle Avoidance

Forces coming from walls, avoiding going through them.



Leader Seeking

The flock's leader has a stronger attraction weight than others.

Inertia Preservation

Stabilizes steering and gets rid of randomness.

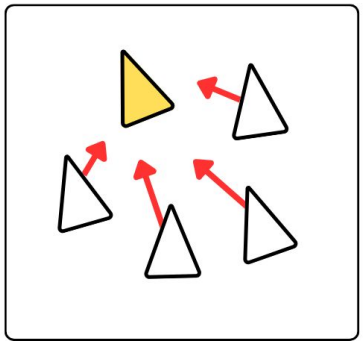
Additional Forces

Obstacle Avoidance

Forces coming from walls, avoiding going through them.

Leader Seeking

The flock's leader has a stronger attraction weight than others.



Inertia Preservation

Stabilizes steering and gets rid of randomness.

Additional Forces

Obstacle Avoidance

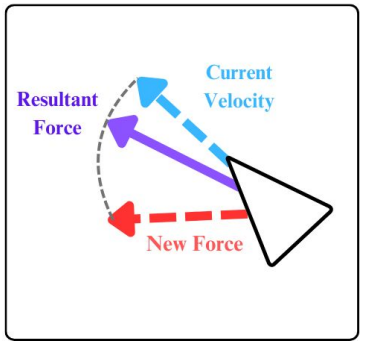
Forces coming from walls, avoiding going through them.

Leader Seeking

The flock's leader has a stronger attraction weight than others.

Inertia Preservation

Stabilizes steering and gets rid of randomness.



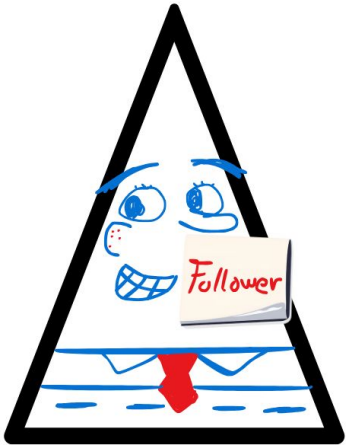
Hierarchy

Leader

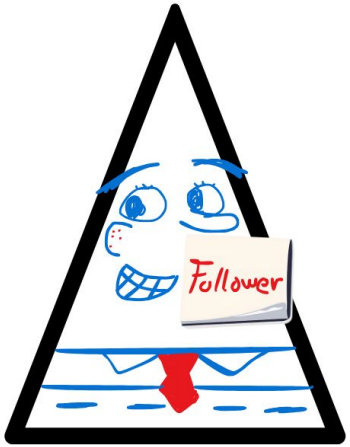
Uses Pathfinding
Not affected by flocking forces
1 per group

Followers

Main movement relies on flocking forces.
Many per group

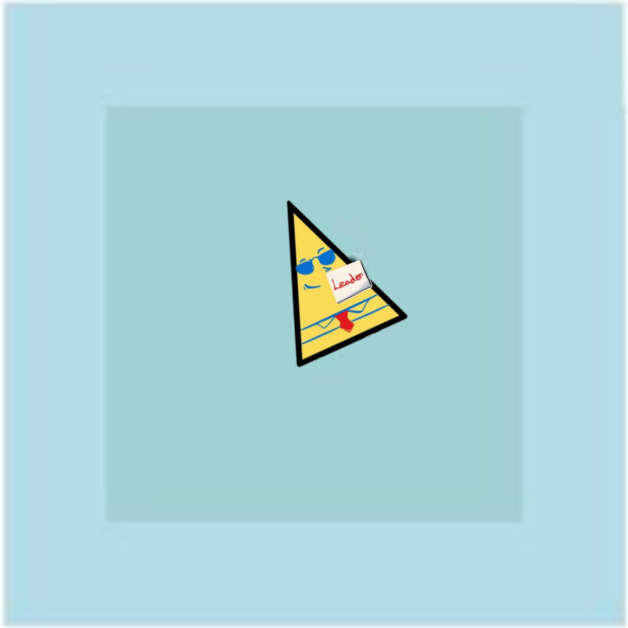


Group Navigation

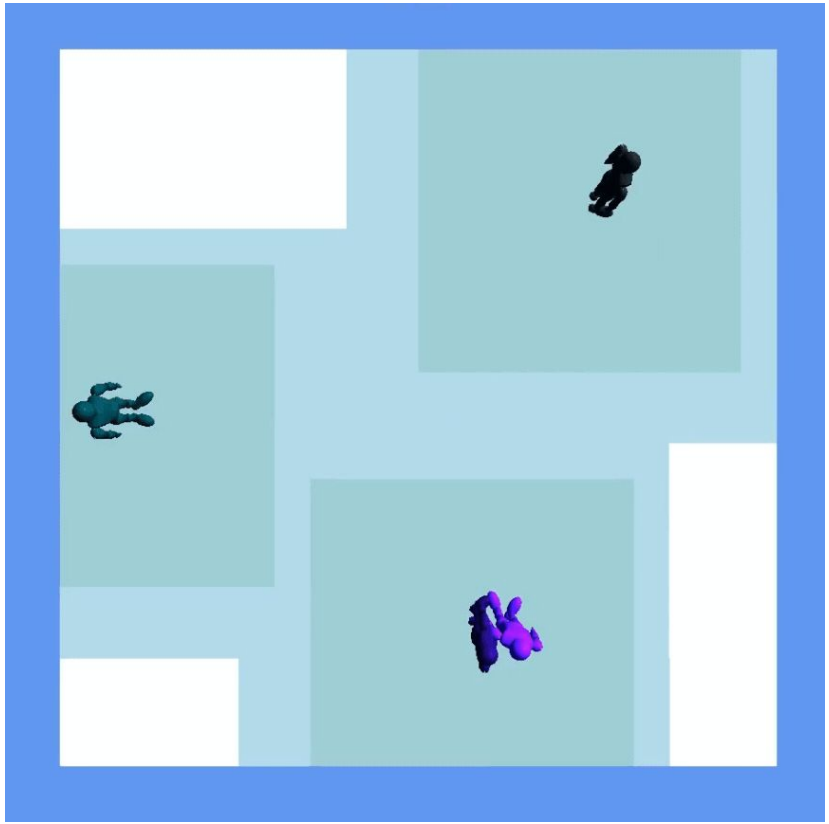


Inside the **inner** radius they will act as a flock

Inside the **outer** radius, they perform a short pathfinding to get to the leader again

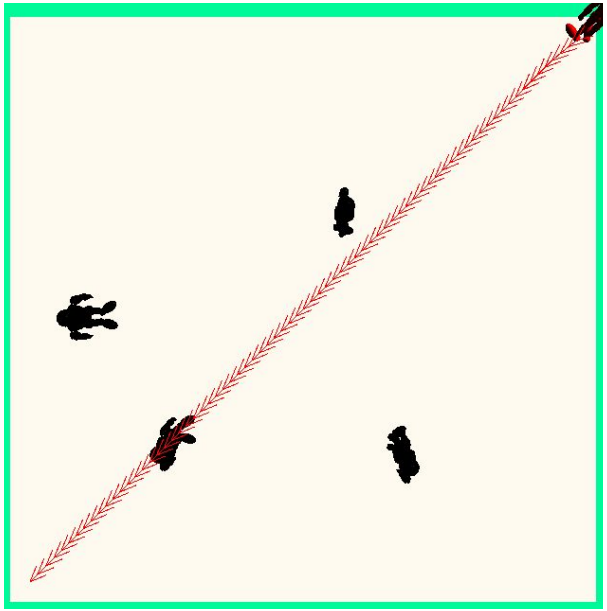


Group Creation

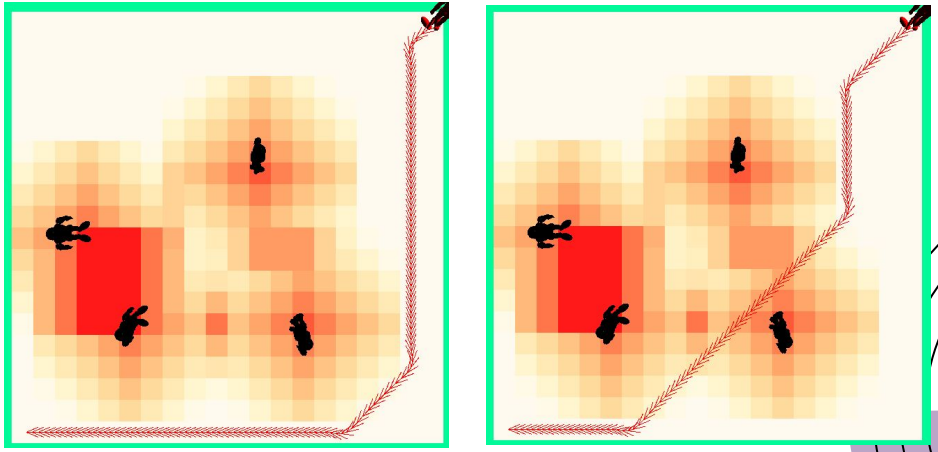


02 Congestion Map

Problem



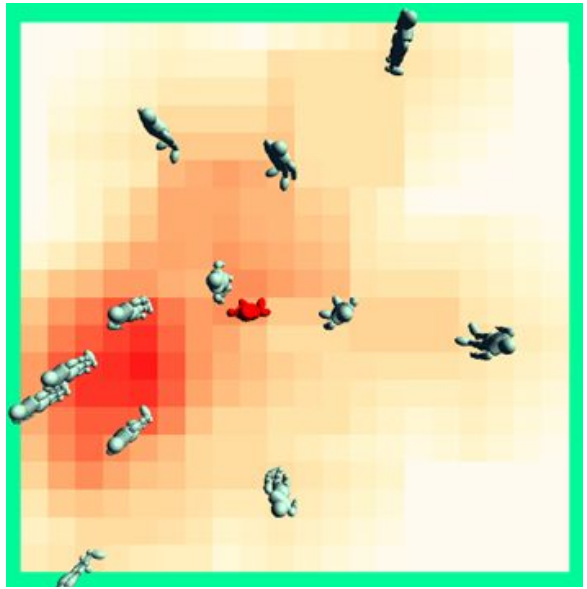
Shortest path



Avoid crowd

Congestion map

- Stores the information of the aggregate crowd density
- It is used to avoid crowded paths
- Updated dynamically



Higher density Lower density

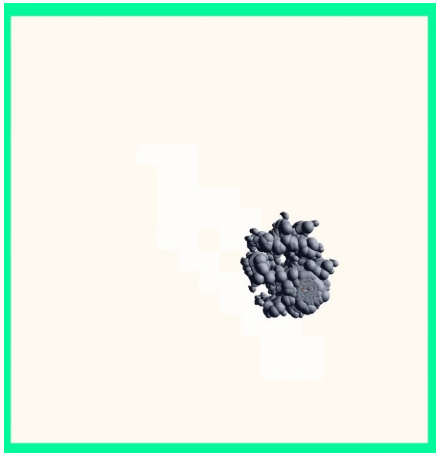
03 Flow Fields

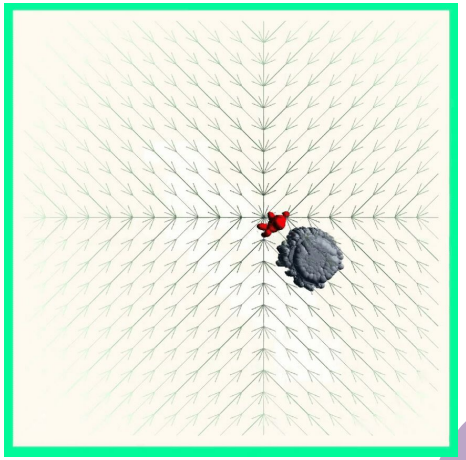
Why?

When the agents move to a certain position some redundant operations are done. Each agent performs pathfinding in each frame in order to find the best path to arrive to the desired position.

A lot of agents = poor performance

A vector flow field store in each cell the vector directed to the neighbor that is closest to the goal. Flow fields are only recomputed when the goal position changes.

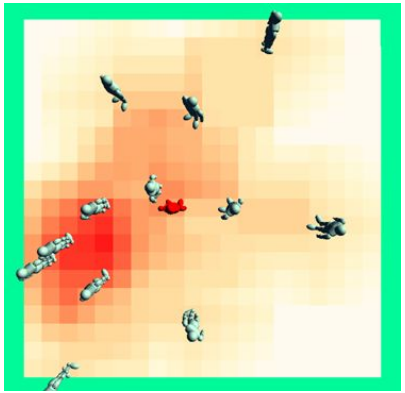




Vector flow field computation

Cost field

- More cost = Area to avoid
- Used to calculate the integration field

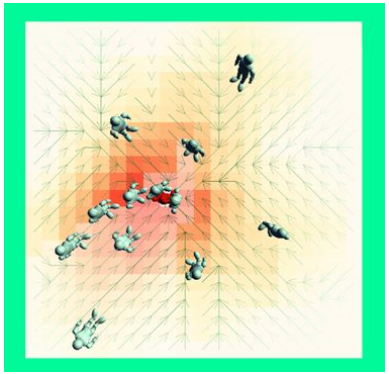


Integration field

- Stores the minimum cost of each cell to arrive to the goal position
- Calculated with dijkstra

Flow field

- Calculate the vector that corresponds to each cell.
- Vector to the neighbor closest to the goal



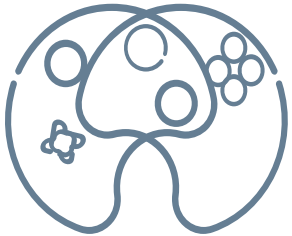
04 Optimal Cover

What is Optimal Cover Placement?

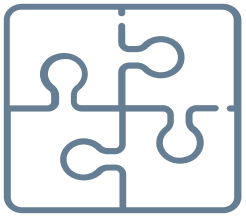
Optimal Cover Placement

From Game Experience and Level Design perspective

Game Experience

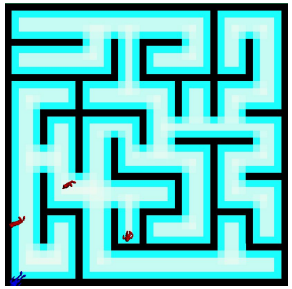


Level Design

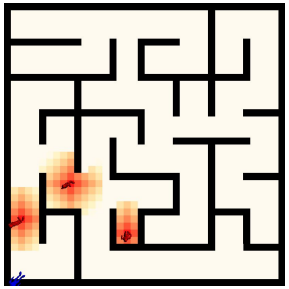


Creating an Optimal Cover Layer

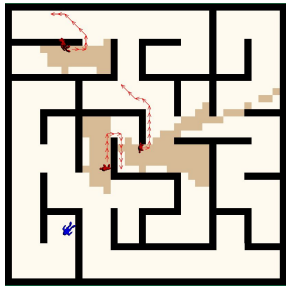
Openness



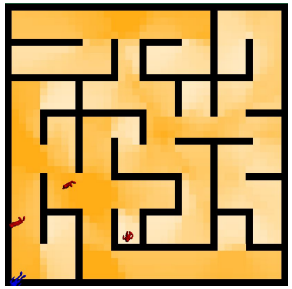
Congestion



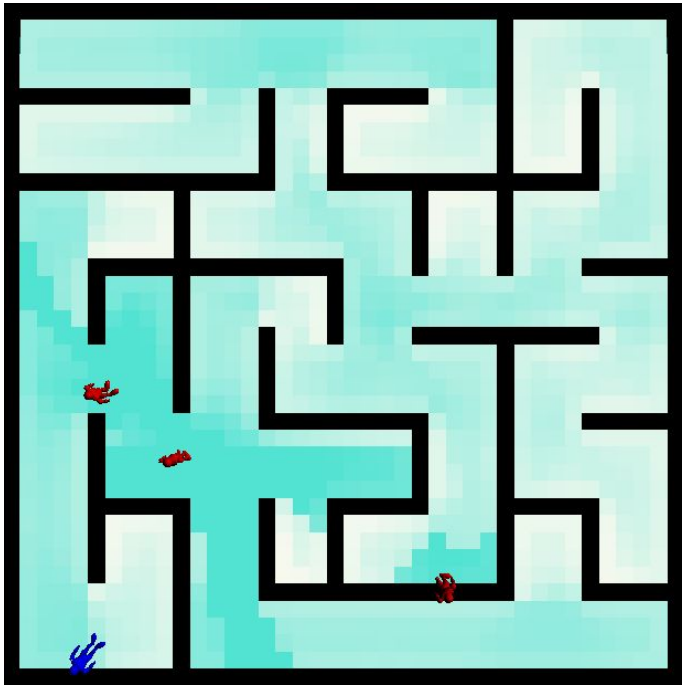
Enemies FOV



Visibility



RESULT



Finding an Optimal Cover

1. Naive
2. Dijkstra
3. Congestion

Finding an Optimal Cover

1. Naive
2. Dijkstra
3. Congestion

Finding an Optimal Cover

1. Naive
2. Dijkstra
3. Congestion

Finding an Optimal Cover

1. Naive
2. Dijkstra
3. Congestion

05 Conclusion

Thanks